



3-Day Certificates

Description: A look at Microsoft's Azure cloud code signing. California implements DROP, global data broker opt-out. Where's the town of "Whata BOD," Idaho? iOS built-in Mail app worked itself out of a job. A 30-minute tutorial for non-coders about AI coding. Claude Code appears to be winning over the AI coding world. Various listener musings on code signing. A bit of magnesium feedback. What use are three-day code signing certs?

High quality (64 kbps) mp3 audio file URL: <http://media.GRC.com/sn/SN-1060.mp3>

Quarter size (16 kbps) mp3 audio file URL: <http://media.GRC.com/sn/sn-1060-lq.mp3>

SHOW TEASE: It's time for Security Now!. Steve Gibson is here. We're going to take a look at code signing. Crazy. We're going to find out what Whata Bod, Idaho has going for it. Steve's going to talk about these three-day code signing certificates. And I'm going to give you a little demo of Claude Code. I've been using it to write applications. All of that coming up next on Security Now!.

Leo Laporte: This is Security Now! with Steve Gibson, Episode 1060, recorded Tuesday, January 13th, 2026: 3-Day Certificates.

It's time for Security Now!, the show where we cover the latest in security news. We attempt to protect you and your loved ones from bad guys in the outside world. And we even talk a little bit about TV shows, books, and vitamins with this guy. It's whatever Steve's into, frankly. Mr. Steve Gibson, hello, from GRC.com.

Steve Gibson: Great to be with you again for, well, I'm not superstitious so the fact that this is the 13th is just fine with me. It's not a Friday. That would be worse.

Leo: It's Tuesday, sir. Yeah, that's not a bad - now, I did have bad luck yesterday. Did you hear me talk about this on MacBreak Weekly? I got phished yesterday.

Steve: No. And it's funny because I forgot to mention it, it's not in the show notes, but I saw - I got a phishing text which said that, what was it, Amazon, it was supposedly from Amazon, saying that the quality of something that I'd ordered did not meet their standards, and so they were giving me a refund, click here. And it was a +91 something something, you know, phone number. So it was like, what? That's not - but, I mean, again, I was like, they kind of had me almost.

Leo: Well, I got had. And I got some insight from it, so it wasn't a complete waste of three credit cards. I got a text from T-Mobile saying your points are about to expire. If you'd like to use them, click this link. I really didn't pay attention for two reasons. One, I'm a T-Mobile customer, and I get a lot of promotional texts from them. And this company's got to knock this off because they're setting their users up. Oh, yeah, this is - I see this all the time.

Steve: Oh, you mean the legitimate companies need to stop this behavior.

Leo: Yes. T-Mobile should not be texting me with promotional stuff legitimately because it sets me up for phishing that looks exactly the same. Now, if I'd noticed the link was to luosa.cc, t-mobile.luosa.cc, I probably would have been smart enough not to click on it. I clicked on it. It said, hey, look, you've got - boy, you've got a lot of points. You could get an iPad. You could get some headphones. What would you like? I said, wow, I don't really need any of this, but these headphones I could give as a gift. Clicked the link. All looked very legitimate. It said, well, okay, we need your credit card. It's free, but there's a 99-cent shipping charge, so we need your credit card for that.

That should have been - then I clicked the link, and it took a long time. That was the real giveaway. So I'm waiting. Put in my credit card information. It said, okay, they're going to send you a text. Got the text. Took a long time because there's a man in the middle, right, waiting for that text so that they can get the credit card number and the text and authenticate it. And what they did, which was really interesting, the text said "To add this credit card to your Apple wallet," add in the number. And I should, I mean, there were so many red flags.

Steve: Again, I mean, I'm sure everybody listening understands that, you know, our guard is down briefly. Maybe you're distracted.

Leo: It was. It was early morning. I wasn't paying attention. And there was urgency; right? These are going to expire.

Steve: Yup. Yup.

Leo: And it was a nice, you know, nice set of Sony headphones. I thought, well, that's pretty good. So I entered the first credit card, said no, this one didn't work. Oh.

Steve: Whoa.

Leo: Entered the second credit card, no, this didn't work.

Steve: And they'll just suck them out until you finally say...

Leo: Three credit cards before I went, wait a minute. Hold on there, buddy. Fortunately, the first credit card was an Apple credit card, which Apple is great. You

go and you say, make that number no good, give me a new one. And that's it, it's done. The other two I had to say to the bank, I need a new credit card. I had to say to American Express I need a new credit card. There'll be a little pain in between reconnecting stuff, which I deserve fully. And in fact shortly after, you know, as soon as I realized it I...

Steve: But you're indemnified from any charges?

Leo: Well, yeah. And as soon as I realized it, I immediately stopped all those credit cards. So I was not going to get bit. They don't - they're quick. So that was the interesting thing. I thought it was smart of them to put it into a wallet. So they put it into an Apple wallet because that's anonymous. You can then use it anonymously at a store, and the store doesn't know who you are. And Lisa called down to me about an hour later saying, "Did you just charge 500 bucks worth of stuff at Lowe's?" I said no. She said, "Well, American Express blocked it." I said, "Well, good." And so I haven't seen any others because they're all blocked. Yeah. Less than an hour for them to get the credit card numbers, authenticate it, add it to a phony Apple wallet, which they own, and use it.

Steve: And Leo, just imagine how many people are being caught. I mean, you're as aware as you could...

Leo: I of all people.

Steve: You're as aware as you could be, but still. And again, even I, like I looked at that text, and I thought, huh, that's interesting. You know? And, but, I mean, whereas normally nothing would have happened, and I just - but then I looked at the phone number, it was +91. I thought, I don't know where that is, but it's not Amazon. So...

Leo: I feel so dumb.

Steve: Well, I've told this story before. I had my main credit card, I could not buy - I could not purchase gas. It was so frustrating because - and it would kill the card if it approached a gas tank, a gas pump, because it turns out that's what bad guys do.

Leo: That's how they use it, to validate it, yeah.

Steve: Exactly, when they get hold of a card, yup.

Leo: Anyway, you know, I'm tempted to never tell anybody that this happened. But I think, on this show especially, I think it's important to say this because we're all vulnerable.

Steve: These are not hypothetical issues that we face. I mean, and this actually will be what you and I will be talking about at our presentation at ThreatLocker at the beginning of March, is I titled our talk "The Call Is Coming from Inside the House."

Leo: Yeah, that's true.

Steve: Because, I mean, that's - it is, that is the threat now. And it is the messiest, least easy to deal with, most pushback from your own employees and staff are all the things you have to keep them from doing in order to protect themselves, protect your organization from inside. So anyway, we have a great podcast today. Maybe it is the 13th. This didn't happen this morning; did it?

Leo: No, it was yesterday. But fortunately it was on a day off so I had time to fix everything.

Steve: So we've got Security Now! Episode 1060, which I titled "3-Day Certificates," which was inspired by a blog post that my continuing poking around in the code signing world led me to that I'm going to share. So we're going to take a look at Microsoft's Azure cloud code signing. The topic we opened last week, boy, it turns out, Leo, a bunch of our listeners are in enterprises where they need to be signing code. And so last week's topic had an extremely high resonance and relevance for them. We're going to talk about that some more. Also California's implementation of DROP to provide global data broker opt-out is interesting. I've got some details about that. Actually I did it, also. I don't know if you have. I did. Also where's the town Whata Bod, Idaho? We're going to look into that. Also I discovered...

Leo: Whata Bod.

Steve: Whata Bod. iOS's built-in map app worked itself out of a job for me. I'm going to explain the back story there.

Leo: Oh, hmm.

Steve: We've got a - I found a 30-minute tutorial for non-coders about how to get into, how to get started in AI coding, like how to ask the questions right, which I want to share with our listeners. Also the fact that Claude Code appears to be winning over the AI coding world.

Leo: Oh, yeah.

Steve: I'm going to share two pieces of information about that and then have you tell us about your own recent experiences, which I have a - I got some sense for. We've got a bunch of listener musings on code signing, a little bit of magnesium feedback, and then we're going to take a look at what use could there possibly be to three-day certificates. I mean, like it barely gets off the ground, and it's landed. So yeah. And of course we've got a great Picture of the Week for everyone.

Leo: Love it.

Steve: So I think another great podcast for us.

Leo: Love it. Okay. Let me tee up the Picture of the Week.

Steve: So I gave this Picture of the Week the caption "It would be funnier if it didn't ring so true."

Leo: Oh, dear.

Steve: "Instead, it's rather sad."

Leo: Yes, okay. This is a FreeRangeComic. I'm looking at it right now. Let me show it full screen, so you can read the entire caption.

Steve: So we have a neat-looking couple of hikers. She's got her little fanny pack, and he's got a walking stick, and they're on a path clearly in some park. And they've come to a ranger who's stepped out of his booth. The arm is down on the gate, preventing them from moving through. And he's holding up his hand, saying "Hold on, stop," pointing to a kind of a billboard-sized screen which is off to the side of the path. And on the screen we see it says "Content loading," with the little spinning thing; right? And he is seen to be saying "Hold it right there, folks. Before you can view any more scenery, you'll have to watch these ads and take a brief survey." So as I said, yes, it would be funnier if...

Leo: So true.

Steve: ...it didn't ring so true. The, you know, even nature is being commercialized, and you're needing to be made into the product yourself if you're wanting to do any communing.

Leo: No communing allowed.

Steve: No communing here. Okay, so based upon the feedback I've received, as I said, over the past week, we appear to have hit it out of the park with our first podcast last week of 2026. I received a bunch of feedback about each of the major topics we covered, and no one complained about my spending time sharing what I'd learned firsthand about magnesium. In fact, many of our listeners want more. So from time to time, you know, again, this is not going to be the nutrition podcast. But again, we're all together, all, what, 100,000-plus of us aging as a group. And we've been at this for 21 years. So we're getting there.

I was gratified to find a great deal of unity over what's going on in our industry regarding the shortening of certificate lifetimes, coupled with the concomitant rising costs of code signing. Since last week's three-hour podcast, which couldn't have handled any more content, I stumbled upon a terrific blog post that was so on point that I want to begin with it this week, much as I began with the same topic last week, by looking at in this case a different aspect of code signing.

The guy's name is Rick Strahl. His post was this past summer, on July 20th. And he tweeted a few days before that, and I'll share that in a second. But he posted July 20th, 2025 from Hood River, Oregon. He gave his posting the title "Fighting Through Setting Up Microsoft Trusted Signing." And while I share what Rick wrote, please keep in mind that no matter how much this guy may sound like me and may be echoing my recently expressed sentiments, this is really his own original writing. So, you know, he's further evidence, I guess, that I and our many listeners who've expressed an opinion are not alone and are not off base in raising an extremely skeptical eyebrow at the recent changes that have been occurring, and which will be adversely affecting everyone who wishes to author code going forward.

So here's what Rick wrote. He said: "So it's that time of year (actually, the time of several years) to renew my Code Signing certificate. I always dread this because it's a manual process; and, invariably, if you're not intimately familiar with the complexities of public key cryptography, the terminology is enough to drive you batty. It's gotten easier since I made some decent notes the last few times I went through this.

"But all that's out the window this time around because the Code Signing rules have changed. Drastically. It actually happened a few years ago, but I was lucky and got my local, still-exportable certificate just before the rules changed so I was able to freeload for at least nearly three years on the old certificate plan.

"The new rules don't allow for locally stored, exportable certificates. Instead, certificates have to be served from one of a few certified online authorities, or the certs must be stored in a FIPS 140-2 Level 2+ compliant hardware security module. The keys cannot be exportable, so they effectively cannot be copied and stored or used elsewhere. So you got the option of server provided keys or hardware keys.

"The idea behind this is to stop keys getting jacked and being used by the non-originating organization. So the new keys are one-time generated and non-exportable, so that they are much more restricted. Online services issue certificates that are good for only a few days when you can use them to sign with, and then automatically roll over to a new certificate.

"What all this means, the complexity of getting a certificate has gotten exponentially worse, and along with that prices have gone up significantly. Base non-EV certs run in the \$350-500 range with fully verified EV certificates starting around \$500 per year. What used to cost me \$180 for three years, the same provider now wants nearly \$1,000 for." He says: "Yikes! It all seems like a huge grift."

Okay, now, in his posting, Rick, as I mentioned, then quotes a separate tweet which he had posted two days prior to this blog posting. On July 18th, Rick posted to X, he said: "As it is, the whole Code Signing thing has turned into another scam of enshittification of a captured audience. If you're publishing software or even packages on NuGet now, you pretty much have to have a code signing certificate. Certificates that used to be \$100-150 (or less for multi-year certs) per year a few years ago now cost 300-400 for basic certs. The EV certs start at \$500 and go up from there. The validation rules for businesses have not changed, and you would think most of the expense is all in that. But this isn't about security. It's about gatekeeping and just one more hurdle for a small business to have to jump over."

So that was his tweet. Then he continues, turning his attention to Microsoft's Azure cloud signing solution. He writes: "Microsoft Is in the Game, Too. Microsoft who requires these code signing rules in the first place for Windows SmartScreen validation and also for other things like NuGet packages, is also providing an Azure service called Trusted Signing to provide Code Signing services. So they're on both sides of that transaction. Create the problem, provide the solution. To their credit their pricing is much better than

what most traditional SSL Cert providers are now charging. Azure Trusted Code Signing is still in preview; but then again, it's been in preview for well over two years. But it looks like what you see and can sign up for now is in the final stages before going to a proper release as a service.

"One reason to look at Microsoft's solution, despite the potential pain and suffering," he writes, "is that the pricing is quite good (as of the time of this post)." And then he has a little chart. The base price monthly is \$9.99. The premium, as opposed to basic, per month is \$99.99. The quota, as in maximum number of signatures per month for the basic \$9.99 is 5,000 signatures per month. Then an over quota is half a cent per signature, so \$0.005, you know, half a penny per signature once you've gone over 5,000 per month. For the premium plan, which is basically \$100, \$99.99, you get 100,000 signatures per month and then the same half a penny for each of the signatures over that.

So he said: "These are non-EV base certificates." Oh, so that means that the difference between basic and premium is not signature quality, which makes sense, right, because we know you don't get any benefit anymore for EV from Microsoft, so why charge more for it? But it's quantity of signatures. So for 5,000 signatures for \$10 a month, basically for 10 times that fee, \$100 a month, you get 20 times the maximum number of signatures before you start having to pay per signature. You get 100,000 signatures.

So he says: "These are non-EV, base certificates that only do basic vetting. For fully vetted EV certificates you'll need to look elsewhere. This pricing, which ends up at about \$120 per year for the single cert, is cheap compared to most of the SSL vendors, most of which start at around \$300 for certificates with mailed hardware keys." Meaning postal mail. They send a key to you. Then you plug it in, and you're good to go. He says: "So, you've got to give Microsoft credit here for keeping costs down and providing reasonable pricing.

"The certificates issued by Microsoft are very short lived, with expirations that last only three days to aggressively thwart invalid signing attacks in case a certificate is compromised." Thus the title of today's podcast, "Three-Day Certificates." We're going to look at the mechanisms behind that.

He says: "Doing a bit of research, out of all the bad options out there, Microsoft's Trusted Signing seems like the least bad solution that's also cheaper than traditional certs from various SSL vendors. The good news is that it works, and pricing is reasonable. The bad news: I wasted nearly an entire day trying to get it to work. Hopefully this post will help you reading this" - so he means those of you reading this - "not to waste quite so much time."

Then his next section he titled "Navigating the Azure Jungle." I'm not going to go through it all, but I'm going to touch on the beginning of this. He said: "If you end up going the Azure Trusted Signing route, plan on having to wade through the Azure dependency jungle of setting up several resources and trying to understand what all the mumbo-jumbo Azure jargon amounts to. If you're doing Azure all day, then much of this infrastructure dance will be familiar to you. But as someone, me," he wrote, "who only occasionally jumps in for some very specific services like Trusted Signing, it's incredibly painful to deal with Azure security, and the Resource dependencies, and the endless nesting of services with badly defined and overlapping naming boundaries.

"For Trusted Signing, finding documentation via search engines was hit or miss. The docs for this are buried behind deeply nested links, perhaps because it's still in or just out of preview," he says, "(even that's hard to tell since some prompts show preview, none of the headlines do)," he said, "and also because previous releases of this technology used a completely different publishing pipeline through the Azure Key Vault." He says:

"There's official documentation, although it took me a bit to discover it." And he put a link in the blog posting, and I copied that link into the show notes. So that's there.

He says: "This has everything you need, but the instructions require some, uh, interpretation. The tools are terrible, and the docs don't make working with them a lot easier by making you figure out where to find files and dependencies and how to install tools. Don't believe your lying AIs," he wrote. "In this day and age of AI assistants and chatbots you would think that things like Azure configuration instructions for setting up an Azure task would be readily available. Heck, there's even an Azure-specific Copilot model that you can use from the VS Code Copilot integration.

"But that actually yielded surprisingly bad results and did not work well with Trusted Signing, either for setup or for the signing part. Part of this might be because Trusted Signing is still in preview, or because the documentation for this is almost non-discoverable, and because things have changed so much with the tooling. Long story short: After a very pissed-off day of going down many wrong paths, I managed to get Trusted Signing to work for my projects, and I'll try my best to provide the details how I have this set up, hopefully sparing a few of you all the pain I ran into."

Okay. And at this point I'm going to stop, almost. So this is about the first 10% of Rick's entire blog posting. Throughout the next 90% of his posting he painstakingly and charitably details the entire process of setting up Microsoft Azure Cloud Code Signing. I've got a link to his detailed instructional posting in the show notes, and I also gave it a GRC shortcut, just to make it easy for everybody to find: grc.sc/codesign, all one word. [Grc.sc/codesign](https://grc.sc/codesign) will bounce you over to this blog posting of Rick's, where you'll see the first 10% is what I just shared, and the other 90 are, like, how he solved the problem.

He finally wraps up this terrific setup walk-through with a summary that's also worth sharing here. As you'll hear, some of this assumes that by now, by the time you've gotten to here, you've managed to slog through everything that he wrote which preceded it. So he sums it up by saying: "The process to set up Trusted Signing was way harder than it should have been - in fact, the entire process took me the better part of an entire work day. The server process is complicated primarily because the nomenclature is so crazy confusing, and the dependency management on Azure is such a pain in the ass. The missing rights from the account to create an identity is particularly maddening, and how you fix it is even more so. But it wouldn't be Azure if you weren't cursing the thing every step of the way.

"The signing process is also a pain in the ass with three different tool chains required. The fact that an Azure Trusted Signing command line interface add-in exists, but doesn't actually support signing, is just ridiculous. With all the resources that are thrown at Azure, it seems petty to not support the one feature that everybody is going to need without having to jump through hoops of managing several tool installation instructions.

"But somewhat grudgingly, I have to say that at the end of the day the process works, warts and all. Microsoft's comparatively lower pricing for the service compared to others maybe makes it worth it; and frankly, the fact that I have my cert running as a service that hopefully doesn't ever need to be updated unless I quit the service is enticing. Yeah, it costs more than it did last time around. I'm now paying almost as much every year as I used to pay for three years. But given the circumstances and the enshittification that now surrounds the entire Code Signing process, this is the best we can do for now. I'm hoping writing this up is helpful to some, and that these instructions won't be obsolete in a few short months because Microsoft changed their designs again, as is so often the case.

"Despite that I finally got it to form, one would hope they fix its performance." Maybe he meant "to perform." Oh, yeah. "Despite that I finally got it to perform, one would hope

they fix its performance." And he said: "Five to eight seconds per file to sign, with no parallelism for multiple submissions, is bad."

Leo: That's painful.

Steve: "And could we" - yes. Like you just sit here waiting for eight seconds for this thing to sign a file. And apparently lots of companies are signing, they have, like, heavy signing burdens. He said: "...with no parallelism for multiple submissions. And could we please have self-contained tooling for signing? For heaven's sake," he wrote, "provide ONE tool that can handle the signing process in one pass without having to install 50 other things. Or better yet, have it built into the Azure command line interface with the trusted signing add-in that's already there. One can hope some of this is due to the relative new-ness of Azure Trusted Signing. But we shall see."

So Rick's blog system supports reader comments, and that posting back in July generated a bunch of feedback. I'll share the first one of many, which followed up and posted: "I'd just like to say that I've been reading your blog for probably 12 years now, and I also went through this. I've learned to parse Microsoft documentation as if I were a machine, and it's nice to know that someone else is turning into the cranky old man of developers."

"I feel like something has been lost from the time we were excited it all worked." Meaning, you know, computing. This guy wrote: "The days where you could slow down the genie effect on Mac with the shift key just to stare at it, when connecting to a system seemed like magic. Now we deal with artificial gatekeeping, auditing, roadblocks, deprecations for seemingly no productive reason. What happened to the joy of being excited that it all worked?" He finished: "Keep on trucking. But also, get off my lawn."

So, yeah. It's not just me and many of our listeners who have sensed that what's happening here is not for the benefit of the world, but for the enrichment of a very few large gatekeeping bureaucracies.

Leo: Yeah.

Steve: Now, I should say that after last week's podcast I did some additional scouting around, and I found that the FastSSL brand offers a standard, like Microsoft, non-EV code signing certificate, if you buy three years, for \$129 per year. And now we're talking hardware. So since it's still possible before March 1st to obtain a three-year plus three-month - remember 39 months - certificate. That's what I plan to do. FastSSL certificates are available from the site cheapsslsecurity.com. As I said, that's what I'm going to do. There's no longer any apparent benefit from obtaining and wielding extended validation certification. Microsoft doesn't even offer it because they don't care.

And since obtaining it means paying a lot more, that is, EV, paying a lot more after first being subjected to basically a full body cavity search in order to qualify for EV, my next code signing certificate will be the bottom-of-the-barrel FastSSL brand. That one will take me from when I get it, which will be late next month, late February of 2026 through to late May of 2029. And at that point, and that means installed in my little hardware USB dongle so I can sign as much as I want to, actually my server will be signing, as everybody who buys a GRC product has code signing on the fly of their own executable, that stuff all got worked out when I talked about it a couple years ago.

So three years from now, May 2029, we don't know what shape the world's going to be in. We don't know what else will have changed. They may have further shortened certificate lifetimes. There may be more pressure in the cloud. Maybe some competition will have stepped up to offer a better deal. We don't know.

So anyway, I've got a link to, for anyone who cares, the FastSSL code signing certificate. It's \$387 for the three years, so \$129 per year. You get to install it into an existing dongle that you probably already have if you've already been doing code signing for the last two and a half years because it was all already dongle-ized. And that's what I know, Leo.

Leo: Unbelievable.

Steve: Yeah.

Leo: I actually, it's funny because - and we're going to talk about it a little bit later. But as you know, over the weekend I wrote some of my own code, and I just moved it over onto this machine so I could show you. And the Apple operating system said "You can't open that. It's not signed."

Steve: Yeah. It is, I mean, it really - it's astonishing, essentially. I mean, you can understand what they're trying to do; right? They're...

Leo: Oh, yeah, for security, I get it, yeah.

Steve: Yeah. Except that bad guys are signing their bad code. Because they're able to protect, I mean, we're hiring North Koreans. We obviously aren't good at figuring out who people are.

Leo: It's an imperfect system. So why enforce it, is what you're saying.

Steve: Yes. And that's, you know, that's what I'm beginning, and I'm guessing this is the old man "get off my lawn" thing. I'm seeing more and more examples of where trying to fix the last 5% percent is creating 95% overhead. You know, again, it's like we need to protect some endangered rodent somewhere in Sacramento, so we can't run light rail through that area without all kinds of environmental exceptions and permits and things. And as a consequence, we don't have any good transportation in California.

Again, I get the intent. But sometimes you end up, it's a case of being your own worst enemy. And in this effort to squeeze, to try to use technology to go all the way to 100% no malware, first of all you're going to fail. You know, even goodware has bugs is the point that I made. And the fact that it's signed doesn't mean that it doesn't have remote code execution vulnerabilities. It just means you know who made it. Well, you pretty much know anyway. Ugh.

Some good news, and a nice acronym. The acronym is DROP, which stands for Delete Request and Opt-Out Platform. Ars Technica's headline was "The nation's strictest privacy law just took effect, to data brokers' chagrin," with the subhead "Californians can now submit demands requiring 500 brokers" - I don't know where Dan got 500. I got 170. But

still, 170 - "to delete their data." So this was written by Dan Goodin, Ars Technica's security guy and technical guy.

He wrote: "Californians are getting a new, supercharged way to stop data brokers from hoarding and selling their personal information, as a recently enacted law that's among the strictest in the nation took effect at the beginning of the year. According to the California Privacy Protection Agency, which is shortened as CalPrivacy, more than 500 companies actively scour all sorts of sources for scraps of information about individuals, then package and store it to sell to marketers, private investigators, and others.

"The nonprofit Consumer Watchdog said that in 2024 brokers trawl automakers, tech companies, junk-food restaurants, device makers, and others for financial info, purchases, family situations, eating, exercising, travel, entertainment habits, and just about any other imaginable information belonging to millions of people."

So the interesting takeaway for me so far, and for us, is to appreciate that this is not passive eavesdropping; right? I mean, these guys are proactively assembling portfolios on individuals. I mean, the more data they get on us by person, the more valuable it is.

So Dan said: "Two years ago, California's Delete Act took effect. It required data brokers to provide residents with a means to obtain a copy of all data pertaining to them and to demand that such information be deleted. Unfortunately, Consumer Watchdog found that only 1% of Californians exercised these rights in the first 12 months after the law went into effect. A chief reason: Residents were required to file a separate demand for each broker." Yeah, okay, what. 500? Or even 170 that I'm aware of. So, wow. "With hundreds," he writes, "of companies selling data, the burden was too onerous for most residents to take on.

"On January 1st" - meaning 2026, a couple weeks ago - "a new law known as DROP (Delete Request and Opt-out Platform) took effect. DROP allows California residents to register a single demand for their data to be deleted and no longer collected in the future. CalPrivacy then forwards it to all brokers. Starting in August" - meaning this coming August - "brokers will have 45 days after receiving the notice to report the status of each deletion request." So it's not just going out into the blue, and you never hear anything back. It's got to be a proactive report of what action they took. Dan said: "If any of the brokers' records match the information in the demand, all associated data - including inferences - must be deleted unless legal exemptions such as information provided during one-to-one interactions between the individual and the broker apply. To use DROP, individuals must first prove they're a California resident."

Dan wrote: "I used the DROP website and found the flow flawless and the interface intuitive." And I'll just add here I did, too, and I'll report on that in a second. He said: "After I provided proof of residency, the site prompted me to enter personal information such as any names and email addresses I use, and specific information such as VIN (vehicle identification numbers) and advertising IDs from phones, TVs, and other devices. It required about 15 minutes to complete the form, but most of that time was spent pulling that data from disparate locations, many buried in system settings."

He says: "It initially felt counterintuitive to provide such a wealth of personal information to ensure that data is no longer tracked. As I thought about it more, I realized that all that data is already compromised as it sits in online databases, which are often easily hacked and, of course, readily available for sale. What's more, CalPrivacy promises to use the data solely for data deletion. Under the circumstances, enrolling was a no-brainer. It's unfortunate that the law is binding only in California. As the scourge of data-broker information hoarding and hacks on their databases continues, it would not be surprising to see other states follow California's lead."

Okay. So I thought that I ought to take this out for a spin, also. Why not? As Dan wrote, and as Leo and I both discovered, it's all out there already anyway, and trusting one more entity who is only asking for my information for the purpose of preventing its warehousing and resale, well, that makes sense to me. So I went over to the new DROP site at consumer.drop.privacy.ca.gov. Again, consumer.drop.privacy.ca.gov. They appear to be behind Cloudflare, since I first encountered that increasingly familiar "Let's verify you're human" intercept page with the little spinning icon doing what it's doing. After a few seconds it finished, and I was taken to the "Delete Request and Opt-out Platform (DROP)," site which identified itself with that web page title.

One of the first things I noted was a "DROP Status" menu item. Clicking that out of curiosity, I was taken to a short page that said: "Enter your DROP ID to check the status of your DROP deletion request. Your ID contains 8 to 10 characters (letters and numbers)." Okay. So that seems sort of cool. You receive a Drop ID which you can use to check back at any time in the future. So I'm thinking that I'll store that in Bitwarden's safe and also alongside my credit bureau credit freeze info, just as a collection of stuff I want to hold onto.

Since this was serious business, I decided that I ought to actually read the "Terms of Use" fine print. And I'm glad I did.

Leo: Oh, I just skipped over them completely.

Steve: I don't blame you.

Leo: What did you find?

Steve: It explains that everything I provide will be forwarded to data brokers; and the more I provide, the better job they'll be able to do of scrubbing me from their systems.

Leo: That's nervous making.

Steve: I know. And it does, you know, it gives you a big gulp. But anyway, so the relevant parts here, it says: "By using the Delete Request and Opt-out Platform (DROP), you agree to the following Terms of Use (Terms) provided by the California Privacy Protection Agency (CalPrivacy, referred to herein as we, us, and our)."

They said: "Use of DROP: By submitting a deletion request through DROP, you consent to disclosure of your personal information to data brokers for purposes of processing your deletion request pursuant to Civil Code section," blah blah blah, "unless or until you cancel your deletion request. Additionally, you acknowledge that data brokers receiving your deletion request will delete any non-exempt 'personal information,' as defined in [another civil code] which pertains to you and was collected from third parties or from you in a non-first party capacity, in other words, through an interaction where you did not intend or expect to interact with that data broker.

"Before submitting a deletion request, you'll be required to verify you're a California 'resident,' as defined in section [blah blah] of the California Code of Regulations as that section read [blah blah]. Verification is made with assistance from state-contracted third-party vendors, including Socure and Login.gov, through the California Identity Gateway. If you're unable to confirm your California residency through these verification service

providers, you may request review of your residency classification pursuant to section [blah] of the California Code of Regulations. You may contact CalPrivacy by visiting this webpage." And there's a link in the show notes for anyone who doesn't want to find it in the Terms and Conditions.

"In addition, you will be prompted to provide personal information, such as name, date of birth, and email address. Certain information is required to verify your residency. Otherwise, the type of information and how much you provide is up to you. However, you must only provide true and accurate information about yourself through DROP. Adding personal information about multiple people in the same request is prohibited." It probably screws things up at the other end, too. "Information received will be used and disclosed to facilitate your request to delete and opt-out of the sale/sharing of your personal information maintained by data brokers registered with CalPrivacy. The more personal information you provide, the greater the likelihood of registered data brokers finding the personal information they maintain about you and deleting that information."

Yikes. Okay. But, you know, it makes sense. If I choose to volunteer the size of my underwear, on the one hand everyone whose business it is to collect and resell such information will have that, authoritatively directly from me. You know, the juiciest and 100% verified information directly from the source that they could ever hope to have. But because this disclosure came through CalPrivacy, its very existence means that disclose or sell it they must not; and that, in fact, they must that information solely for the purpose of identifying me and, having done so, delete it, and everything else they may have previously aggregated over time about me.

The Terms of Use continues: "Data brokers are required to process deletion requests at least once every 45 days beginning August 1st, 2026. Your submission of personal information through DROP is governed by CalPrivacy's Privacy Policy, which is incorporated into these Terms by reference."

And so, under Prohibited Uses, they say: "While using DROP, you agree you will not: Use DROP for any fraudulent, unlawful, or prohibited purpose. Impersonate any person or entity or misrepresent your affiliation with any person or entity. Interfere with or disrupt the operation of DROP, or the servers or networks used to make DROP available, including threatening the integrity or security of DROP. Restrict, disrupt, interfere, or inhibit any other person from using DROP." And finally, "Reproduce, duplicate, copy, sell, resell or otherwise exploit, for any commercial purposes, any portion of, use of, or access to DROP. Violating these terms may, on a case-by-case basis, result in restriction of your ability to access and use DROP."

And then they did add aiding another person with their request, which, you know, could be necessary. They said: "You are only permitted to aid another consumer with their deletion request if that person has authorized you to do so, and you meet the requirements described in [some section]. In addition, the consumer must first have their residency verified as described in the Use of DROP section above. When aiding a consumer with their request, you or the consumer must disclose your full name, email address, and business name if applicable through DROP when prompted." Because you are asked, is this for you or for someone else that you're doing it on their behalf?

"In submitting information on behalf of another person, you certify that you have authorization to do so, and that the information you provide is true and correct. Adding personal information about a person who has not authorized you to submit a deletion request on their behalf is prohibited."

And then finally - anyway, there's a little bit more, but we've got enough of this. Everyone has a sense for that. They do talk about third-party vendors' involvement and their disclaiming their liability over third-party conduct, which is not very comforting. But,

you know, that's what you get anytime attorneys are asked to review and revise anything like this.

So the term ends with something titled "Notice at Collection of Personal Information," which says: "YOUR DATA: When you use DROP, the California Privacy Protection Agency collects personal information you enter such as name(s), email(s), phone number(s), dates of birth, zip code(s), Mobile Advertising ID(s), Connected TV ID(s), and Vehicle Information Number(s). We also collect usage time, device ID, and IP address. We use the data to provide your deletion request to registered data brokers, enhance the product, respond to questions, and ensure safety. Providing information and using this service is voluntary. Do not provide unrequested personal information."

And finally: "YOUR RIGHTS: You may access records with your personal information. Collection is subject to the Information Practices Act and state policy. If you have any questions or concerns about this policy, contact us, [blah blah blah]."

Okay. So I did what Dan did. And it did take about 15 minutes. I chose to use Login.gov since I already have an account setup there. I don't recall why, but the email address they have for me...

Leo: Global entry.

Steve: Yeah. Oh, that could easily be.

Leo: Yeah, that's what Global Entry uses, yeah.

Steve: Although for me, the email address they have for me is the one that I was using in 2018. So it may have been set up for my Social Security stuff in advance of my turning 65.

Leo: It's for Social Security, that's right, yup.

Steve: Yeah. So in any event, after providing my phone number to Login.gov, the site used SMS to send my phone a link. Clicking that link took me to a page which requested access to my camera so that it could manage capturing the front and back of my California driver's license. It did that with a cool Arnold Schwarzenegger Terminator green grid overlay kind of thing.

Leo: [Beeping sounds]

Steve: And for each of the two - yeah, exactly. And for each of the two captures it asked my permission to send it for verification, which took a few seconds each time. After that I was returned to the DROP page where I provided both Steve and Steven as my first names. I avoided, Leo, adding the "Tiberius" as my middle name.

Leo: Probably a good idea.

Steve: Yeah, I didn't want to confuse anything. Then I provided my last name...

Leo: There's a lot of verification going on here. I've got to - that round tripping a bunch of times with this.

Steve: Yeah.

Leo: And you have to verify your email and...

Steve: Right, right. I gave them my last name, my date of birth, my Social Security number, my residence address, my Vehicle Identification Number. There were places to add a mobile advertising identification number and a Smart TV ID. Until Apple refreshes their Apple TV hardware, which I'm just holding my breath for, I'm using Roku. And while Roku does have an advertising identifier, that number is not user displayable without side-loading a Roku channel for displaying such internal stuff, and that was more than I was interested in doing. And I also have...

Leo: Most people wouldn't know that kind of thing at all.

Steve: Exactly. And I do have app tracking turned off in my iPhone, so there was nothing to share there, either. Once that was all complete I was taken to the "Deletion request submitted" success page. And there I received my promised, I was going to say eight-character, but it's actually two sets of four characters hyphenated, so I guess that's nine characters. That's that Drop ID which I can then use to check back on my DROP status at any time in the future, although nothing's going to happen until late August, or actually I guess maybe even early September.

Leo: So much easier to get a DeleteMe account. I'll be honest with you. But okay.

Steve: Although it did occur to me that, yes, but then DeleteMe must be asking the same things; right? Anybody who's going to be - is wanting to delete your data...

Leo: The most you give them the better, that's right, yeah.

Steve: Yeah. Anyway, so one cool thing is that having done this, the DROP page's menu, the main menu on the DROP page added two new page links. One was for "My Data Profile," which was that form that I filled out which was all then viewable with a whole bunch of asterisks, you know, blanking out most of the information, but letting me know like what the last four digits of things were. And the other was the "Data Broker List" pages.

Leo: DeleteMe is a sponsor, I should mention, but it does [indiscernible] got to disclose.

Steve: Right.

Leo: Okay.

Steve: So the "My Data Profile" page shows a ring chart, which is, you know, like a pie chart, but with the center hollowed out, where we are informed that a total of 170 named individual data brokers are registered with the state of California and are thus subject to this new law which, as I said, went into effect on January 1st, with an eight-month grace period. But what's most cool is that, once this happens, the ring chart has categories, you know, that'll be like a pie chart, for Deleted, Opted Out, Exempted, Record Not Found, and Pending. So I'm going to be - we have to all wait, you know, eight months. But I'll be very interested in seeing both the "Deleted" and the "Record Not Found" counts. Currently all of this stuff is 0/170, you know, zero out of 170. So what will they be in September? It's going to be interesting to see how that goes.

The "Data Broker List," that second new page, actually displays the current status of each of those 170 individual data brokers. At the moment they're all currently shown as N/A, and the filter option, which is a column in the table, contains the same itemizations as the ring chart - Deleted, Opted Out, Exempted, Record Not Found, and Pending. So you'll be able to select by those or sort by those, which, again, I think will be very interesting to see. And I'll just say, and we've sort of touched on this several times already, but looking at all of this, I was reminded of what Dan wrote. You know, he said it initially felt counterintuitive to provide such a wealth of personal information to ensure that data is no longer tracked.

As I thought about it more, I realized that all that data is already compromised as it sits in online databases, which are often easily hacked and, of course, readily available for sale. So again, yes, it's somewhat creepy to be volunteering all that information, you know, providing it to the, like, indirectly to the trackers who have been doing all of this, whose business it is to do this. But we can presume that only a tiny fraction of Californians are actually going to even know about this or take the time. It would be nice if it were a big groundswell. But I doubt that's going to happen. And as we said, Leo, even using our sponsor DeleteMe, you've got to tell them all this in order for them to tell the bad guys what they have to delete.

Leo: Right, exactly, yeah.

Steve: So no way around it.

Leo: So you saw how many data brokers? Because I'm only seeing 89.

Steve: Oh, I got 170.

Leo: Well, you are a lucky man.

Steve: I don't know why. Yeah, sure enough.

Leo: Isn't that weird?

Steve: Yeah. I wonder. Let me go click on mine.

Leo: Maybe they're going to add more over time. We know there are more than 500 in the real world. So, you know, I just feel like - I don't know.

Steve: What?

Leo: It's not going to happen till August?

Steve: Yeah. Nothing happens. On August 1st...

Leo: That gives the data brokers lots of time to lobby.

Steve: On August 1st, the 45-day timer starts.

Leo: Begins.

Steve: Yes.

Leo: Which gives the data brokers a lot of time to lobby our state legislators to change their minds.

Steve: Oh. I clicked on "I Accept," and it made me scroll down to the bottom of the Terms of Service.

Leo: That's insane.

Steve: Even though I already said - oh, and now I've got to log in. And, okay, I'm not going to - I wouldn't mind if you do that during the podcast.

Leo: This is, I think, carefully engineered to discourage some maximum number of users, to be honest. I feel like, see, this is the first state to ever do this. And we certainly have no federal law doing this. And I feel like that the reason is law enforcement doesn't want it. They love this information. Marketers have big checkbooks to write to congress critters.

Steve: And it's what runs the Internet, unfortunately.

Leo: And it's what runs the Internet.

Steve: It's what finances the Internet, yes.

Leo: Right. You know the FCC just said, hey, by the way, Verizon, you don't have to unlock phones. You can leave them locked. We live in a world now where the people with the pocketbooks dictate the law. It's not the consumers.

Steve: No.

Leo: So I just - I feel like - go through this is probably worth it. We'll see. I'll watch with interest. But I don't have high hopes.

Steve: And I will certainly report...

Leo: And why do you have 159 data brokers, and I only get 89? What hell's that all about?

Steve: Yeah, I got 170. Yeah, in fact you can see in the show notes, that is a picture of my status screen on 01/10/2026.

Leo: I got robbed. Did it get - it was 189 right out of the box, huh?

Steve: No, it's 170 right out of the box.

Leo: 170 right out of the box. Mine is 89 out of the box.

Steve: It was what you see there at the top of page 10 of the show notes.

Leo: Yeah.

Steve: And we compare it to yours, and sure enough, maybe Southern California has got them extra crawling around. I don't know.

Leo: The whole thing feels a little - I don't know. Suspect. We'll see.

Steve: There it is.

Leo: We'll get back to you in August.

Steve: Yes. September, actually, because August, the 45-day timer starts. So it won't be until a little past the middle of September that we're going to actually get some - they have 40 - well, it could be sooner. They have a maximum of 45 days.

Leo: Right.

Steve: So this all lands on them.

Leo: Do you think they're going to rush to do this? It's going to happen on day 44, 23 hours, 59 minutes, and 59 seconds. And right up to that very second they're going to sell it like crazy.

Steve: And it does feel like, you know, it's like there are those of us who have set our browsers to say Do Not Track, and my Global Privacy, my GPC or whatever it is, my Global Privacy Control, you know, it's saying no. Every so often I come to a site that says, oh, we're going to honor your Global Privacy Control wishes. And I'm thinking...

Leo: What?

Steve: ...well, yeah. Well, that's good. Thank you.

Leo: You're the one.

Steve: Okay. Break time, and then we're going to find out, Leo, where is Whata Bod?

Leo: Not Whataburger.

Steve: No.

Leo: Because I know where that is. It's just down the street.

Steve: Two words. It's in Idaho. It's two words, W-H-A-T-A and then the second word is Bod, B-O-D. Whata Bod.

Leo: Whata Bod.

Steve: What about Whata Bod?

Leo: That's what Lisa says whenever "Reacher" is on. But that's another story entirely.

Steve: And, boy, he does like to take his shirt off; doesn't he.

Leo: He's got - yes. Every episode.

Steve: Yeah. Okay. So I wanted to share a wonderful bit of AI hallucination news from this past weekend. The U.S. National Weather Service has withdrawn a wind forecast from its social media platforms after its new AI-powered system generated a map of Idaho which included two fictitious town names.

Leo: Oh, my god.

Steve: "Orangeotild" and "Whata Bod."

Leo: Oh, my god.

Steve: The wind weather forecast map, which was initially shared on social media by the weather office in Missoula, Montana on Saturday, depicted those two nonexistent towns occupying Idaho's Camas Prairie region. The forecast posting helpfully encouraged residents to "Hold onto your hats," indicating that "Orangeotild" faced a 10% chance of high winds, while "Whata Bod" to the south would experience calmer conditions.

Leo: Well, hold onto your bod.

Steve: Hold onto your bod, that's right. Beyond the gratuitous synthesis of those two prominently featured towns, the National Weather Service's map also contained multiple spelling errors and geographical inaccuracies.

Leo: Oh, geez.

Steve: The Weather Service was quick to blame these mistakes on the use of generative AI technology. That's right. Blame the AI. That's our new slogan.

Leo: And nobody checked this, nobody looked at it?

Steve: No. No. No, Leo, because they've all been let go.

Leo: That's the thing, and that's what happens. You fire everybody, yeah.

Steve: I have an interesting adventure to share. Several months ago I began noticing that my beloved eM Client, email client that I've talked about on the show, that I discovered and talked about on the show, had stopped notifying me of incoming email in a timely fashion. Someone would say, you know, that they had just sent something, but after waiting a reasonable length of time nothing arrived. I discovered that by completely closing and then restarting eM Client, then it would again, for a while, be reliably notifying me of newly arriving mail. I haven't mentioned this until now because I hadn't been able to affirmatively verify that eM Client was the problem, though it certainly seemed to be. And, I mean, I was upset. This has been going on for months.

But then a few weeks ago, something, I don't remember now what it was, but something caused me to look at the logs of GRC's hMailServer. What I discovered was that the server had been crashing and restarting, leaving a trail of mini-dump crash log dumps behind. And before the server would crash it would log the source of its pain. It appeared to be something about IMAP and the retrieval of large file attachments. And they were the IPs of my two locations. So that made sense, too.

So I spent a few hours having a heart-to-heart with ChatGPT to see what it might have absorbed and chasing down the various leads that it was generating for me. There really didn't appear to be any reason to suspect that eM Client was behind the trouble, and the hMailServer discussion boards, you know, they were not of any help. They were filled with the typical threads of people commenting without actually knowing what they were talking about. So, okay. Look elsewhere for a solution.

What I did realize was that if eM Client - or for that matter any email IMAP agent that, by nature of the way IMAP works, maintains an open TCP IMAP connection, where it was expecting to be proactively notified of newly arriving email, which is one of the things that IMAP is able to do, you're able to put a connection into an idle status. When the other end of that connection would crash and restart, as the server was, that TCP connection would be left hanging. So eM Client would never receive the news of new email, nor would it know that the connection had gone down if it wasn't, like, proactively pinging for, like, some life at the other end.

So my environment contains both eM Client and a collection of iOS devices, iPhones and iPads. And as I was correlating the times of the server crashes with my own actions, it appeared to be more connected to iOS than to my use of eM Client, which I have on desktops. Some googling revealed that, to my surprise, iOS has historically had a surprising degree of trouble getting the IMAP protocol correct, and this has been a source of great annoyance to those tending IMAP servers before me. The moment I deleted the troublesome account from all my iOS devices, all server crashing stopped. This was about a week ago, and the server has never crashed since. And I even checked, Leo, just during our last break. My eM Client on the desktop has resumed its previous perfect behavior of immediately notifying me of any new incoming email.

So the reason for my having dragged everyone through this sordid tale is that my strong, in fact overriding, proclivity is to "live off the land," right, wherever possible. Since every one of my iOS devices came with a built-in iOS email client, the last thing I would ever consider doing would be installing a second redundant email client. But yes, indeed, things had come to that. I remembered that the eM Client folks offered their mobile clients at no charge, so I thought "Okay, let's give it a try."

I downloaded eM Client from the Apple App Store. Naturally, although I specified exactly the client name I wanted, eM Client was not first in line. No. It was preceded by sponsored apps that were paying to have my search results contaminated for their benefit. Like many others, I'm beginning to feel that the shine is fading from the Apple, which is truly sad.

Nevertheless, I was able to find, download, and run eM Client. The first thing it asked upon running was whether I would like to import my existing world from a desktop instance. I thought "What?!! Yes, please!" So it told me to open any already-configured desktop instance of eM Client. In its menu under "Tools" I would discover "QR Export." Sure enough, my Windows desktop eM Client displayed a massive QR Code which my mobile instance saw, and it was immediately set up with all my accounts, logins, passwords, tweaks, preferences, everything.

So it's been about seven days since I made that switch across all five of my iOS devices. You know, I'm still not accustomed to how much better the mobile version of eM Client is

compared to Apple's built-in, but uninspired, mail client. eM Client even runs on my oldest iPad which I now have to keep plugged in. It's so cold that ChatGPT's client refuses to install, scolding me that I need to update to an iOS version from sometime this century. But, you know, I'm running the latest one that will run on that hardware. So I just use - I use ChatGPT from the browser when I'm on that little iPad. And I am also, not only am I waiting for new Apple TV hardware, I'm dying and hoping for an OLED, you know, new iPad.

Leo: Yeah, me, too. Although that's going to be end of this year. You're going to be waiting a while. Or maybe...

Steve: Okay. In that case I'm going to have to make the, you know, I can't wait...

Leo: Bite the bullet. You know what I bought that I - because I didn't want to wait either. And I'm very happy with it, the new Lenovo, well, this isn't the new one, actually. This is January's Lenovo X1 Carbon with an OLED screen. And it's...

Steve: Yes.

Leo: It's super light. It's really great. I just - I'm madly in love with it.

Steve: I get it. Now, I did something different. I bought a Lenovo little, you know, the small form factor block because...

Leo: Yeah, yeah, yeah. Oh, we talked about this. That's right. You bought the desktop, yeah, yeah.

Steve: I bought the desktop because it can drive three screens.

Leo: Right.

Steve: And then I remember hearing you talk about your laptop, looking at it, thinking, you know, that's gorgeous and everything. I was almost going to pull the trigger when I said, wait a minute, no. I don't ever go anywhere.

Leo: You don't need a laptop.

Steve: So, well, I want to be able to be downstairs, you know, and be socializing with Lorrie, not hiding up in my cave for, like, in the evening.

Leo: Oh, so you - that's why I have a laptop. I don't go anywhere either, but I move around the house.

Steve: I spent \$400. I bought the cheapest...

Leo: Terminal.

Steve: Yes. It's the largest screen, dumbest Windows laptop available.

Leo: It's a terminal.

Steve: It is. Well, because Remote Desktop. I remote desktop to GRC's servers. I can remote desktop upstairs...

Leo: Right. Exactly.

Steve: ...to my machine.

Leo: Yeah.

Steve: So I get all the speed and performance. I don't have to worry about synchronizing everything and all that.

Leo: Yeah, that's smart.

Steve: I just I have a mouse, screen, and keyboard that I can have out on the patio, in the family room, wherever I am, talking to the computer that I left running upstairs. So I think that was - for me, it was the right solution because again, I don't, you know, if I travel, I just take a pad with me, and I'm fine because I'm not actually doing any work.

Leo: Right. Yeah. Actually, and then I did buy this, and at CES they announced the next generation, 14th generation, which has some major improvements. But the OLED is very nice. That's the thing I really wanted was the OLED. And I think you're right to wait for the MacBook. But it's going to be a wait. So maybe as long as a year.

Steve: Wait. Not MacBook. iPad.

Leo: Oh, you wanted an iPad. Oh, they have an OLED iPad. Yeah, the iPad Pro is fantastic. I have the OLED iPad.

Steve: No, I'm sorry, I want a Mini. I like the Mini. It's the right form factor for me.

Leo: Well, you can have any OLED screen you want with a Mini. The Mini's great. I love the Mini.

Steve: No Mini is OLED.

Leo: Oh, you're talking about iMac. iMac has a screen. The Mini has no screen. It's just a NUC.

Steve: I'm sorry, I'm talking about the iPad Mini. The iPad Mini.

Leo: Oh, the iPad Mini. But who's on first? That's what I want to know. Oh, the iPad Mini. No, they don't have an OLED. That's going to come out. But that'll come out sooner than the - yeah.

Steve: Oh, that's what I was thinking.

Leo: Yeah, that'll probably be out this spring, yeah.

Steve: Okay, yeah.

Leo: They try to get those out for the school year. So certainly by June.

Steve: Right.

Leo: Yeah.

Steve: Okay. So I just wanted to say that eM Client is free. GRC's server never crashed, not that everybody else is going to have that problem, but I was. And I just wanted to give everyone a heads-up that whether or not you're running eM client on your desktop, the 100% free eM Client for iOS or Android is truly lovely. And if you are one of our many listeners who switched to the desktop eM Client after my discovery of it, or if you're one of our other listeners who wrote to me rhetorically, asking what took me so long to find the eM Client for the desktop, both groups will get the additional joy of instantaneous account setup by cloning the QR code from your desktop to your mobile device.

Anyway, before I close the topic, I do want to acknowledge that I know someone's going to write to me. There's no excuse for anything some remote email client might do to cause the email server that I'm running to crash. I'm 100% in alignment with that sentiment. But I love hMailServer. It is everything I want in a Windows-hosted open source email server. In addition to many great features that I use, it publishes a COM interface that's allowed me to automate parts of its operation to integrate it into GRC's email system. Because it's open source, I was able to engineer its operation to get it to do exactly what I needed. But since I'm not in the position to spend lord only knows how long it would take to fix its actual problem, I am treating the symptoms, yes.

And in this case, that worked to my advantage since it allowed me to stumble upon eM Client, which doesn't induce those crashes because it's not iOS, which does. And then it turned out to be a more pleasant user experience than Apple's own native Mail app, which I would otherwise have never discovered. I'd be using Apple until the end. So now

I'm glad I am not. I have two pieces that I want to share next. And then, Leo, I want, as I mentioned, you to share a little bit of your recent Claude Code revelations.

Leo: Certainly.

Steve: One of the AI newsletters that I keep an eye on is called The Batch. It's published by DeepLearning.ai. And last Friday, an issue of "The Batch" arrived that caught my eye because I was pretty certain it would appeal to many of the non-coders who follow this podcast. The issue of the newsletter opened with: "Dear friends: We just launched a course that shows people who have never coded before, in less than 30 minutes, how to describe an idea for an app and build it using AI."

They wrote: "It's now time for everyone marketers, product professionals, operations specialists, analysts, students to build software applications with AI." And I know, Leo, that this is singing from your hymn book. They said: "I've often spoken about why everyone should learn to code. I'm seeing a rapidly growing productivity gap between people who know how to code and those who don't. For many job roles I hire for, I now require at least basic coding knowledge. Many times, after I speak with a non-technical audience about the importance of building software using AI, people ask me how to get started. In the past, I didn't have a great answer.

"That motivated the DeepLearning.ai team to create 'Build with Andrew.' It's the best way for someone who wants to try vibe coding to get started. This course requires no prior knowledge of AI or coding. And it's vendor-agnostic. Specifically, learners can use these techniques with whatever tool they're most comfortable with - like ChatGPT, Gemini, Claude, or the chatbot built into the DeepLearning.ai platform."

Okay. So the "Andrew" cited here is Andrew Ng.

Leo: Oh, yeah.

Steve: The founder of DeepLearning.ai - yes, Leo - the website that's hosting this free course. For those who don't know, Andrew also co-founded Google Brain and Coursera and led AI at Baidu. He's an adjunct professor at Stanford University, former associate professor and Director of Stanford's AI Lab, SAIL. So Andrew is certainly not some random YouTube influencer trying to get "Likes." To help everyone find this free 30-minute course, I've created a GRC shortcut using Andrew's first name, so grc.sc/andrew. So just to be clear, I cannot vouch for this myself since I did not take the time to explore it. But Andrew is obviously the real deal, and it would certainly seem worthwhile for anyone who might have been wondering how to take the first step toward AI-driven coding. So grc.sc/andrew, and that bounces you over to build-with-andrew under courses at DeepLearning.ai.

And the second piece I wanted to share is from a listener, Al Liebl, who said: "Hi, Steve. I've listened to the podcast for years and have thoroughly enjoyed it. I currently work in cloud security and find your content informative. Keep up the great work. I'm writing to you because of an open source project I've been working on. I should tell you, I'm a terrible programmer. I'm 54, wear progressive glasses, hunt and peck and likely have mild ADD." And he has a little grin there in his note. "Having said that, I've been around computers since my dad built a Heathkit H89 in our basement, and I learned to use it. And as an adult I've worked for software and security companies in various roles, so I know what good looks like, and with my current role in cloud security I understand what gets attacked and how.

"I've grown tired of the lack of security, privacy, and trust online and decided to start VettID. I've spent a bunch of time creating the design and then tried to figure out how to find people to help me get it built. That failed spectacularly. So I dusted myself off and decided to go with plan B: AI. A few months ago in my free time" - he says, "I work full time" - "I went through some online classes for using AI for coding. They were helpful in teaching me the basics, and I started using ChatGPT. It worked, kinda. I could prompt for what I wanted, and ChatGPT would spit out the code, and I would have to paste it into VS Code and do the things (build, commit, push, deploy, et cetera). In this situation I was still the clumsy bottleneck."

"So I did a quick search and found Claude Code, and it has been life changing!" he says. "I pointed it at my repo and the rest is history. I trained Claude Code on my design and refined it. I had Claude Code develop a plan to implement it, leveraging multiple Claude Code instances - one for iOS development, one for Android development, the main instance as the backend/frontend/lead, and a Raspberry Pi as a tester. The plans were broken into issues assigned to each repo, and the different instances could communicate via issues for troubleshooting. Working part-time over the last few months, I'm close to having the first version done. All the best, AI."

So what occurred to me was, when I read that, and also this really cool "Build it by Andrew," is that, Leo, you had just been saying something similar in the preshow on Sunday.

Leo: Yeah.

Steve: So I thought I would, you know, give our listeners a chance to get caught up with what you have found.

Leo: Well, get ready because it's not just going to be me you're going to be hearing from, or Andrew, or anybody else. This is, I think, going to be a drumbeat. I really feel like that we have turned a corner in AI in general. But what people mostly experience is ChatGPT, a chat interface and that kind of thing, or maybe image generation. The people who are really in my mind most impressed with AI at this point are coders who are using AIs to do code. And I think we've universally coalesced. There've been a lot of choices. For a while, ChatGPT's codex was the best one out there. You know, there are coding models from China, as well, Qwen and others. In fact, DeepSeek's got a new coding model coming out sometime soon that people say this thing is amazing.

But I think most of us have kind of, at least for the time being, centered on Claude Code, Anthropic's Claude. They did a big update November 24th with Opus 4.5. And they've been adding a lot of features since. But they've also been really focused on code and making Claude Code be better and better. And the thing that's really accelerated the development is lately they've been using Claude Code to improve Claude Code. And I've seen a number of people who work at Anthropic say, yeah, most of the stuff we've released has been written by Claude itself, not by us. And that's a big change.

So, you know, I've been using Claude Code with a \$20 - I have \$20 subscriptions to the cheap ones for everybody. Perplexity, OpenAI, everybody, including Grok because I get it for free because Elon's given me a nonconsensual blue check. So I've tried them all. But I was, when I got this new ThinkPad, I started, I set up Linux, and I started configuring it using Claude Code. And instead of me looking up, oh, what's the syntax for this, because I'm using Sway, which is a very text-based,

you could probably use Nix or other things, very text-based configuration as opposed to a GUI configuration. And Claude was great. It knew everything. Said, oh, yeah, let me - can I put an icon up there? Can I make it wider? Can I - and it was doing all that.

I thought, this is pretty good. I use it so much I would start getting to the point where I would say, okay, well, you've used all your credits. You have to wait for a couple of hours now. And it's usually just to kind of like, you have to wait till 2:00. It's usually just a couple hours. But I thought, all right, I'm going to bite the bullet. There's a \$200 plan, and there's a \$250 plan. \$250 plan is 20 times the number of tokens. The context window is huge, it's 200,000K. The bigger the context window, the more it can hold. And I'd say 200,000 tokens is about 150,000 pages of stuff that it can hold in its head while it's doing stuff.

The bigger the context window, I don't want to say more effective because it can maybe be less effective, but the more it can know about at the same time. It's kind of like our own brains; right? In fact, that's one of the problems I had with coding always, even when I was younger and my brain was more adept, is the complexity rapidly got out of - my context window got too big. So, and this is how coders handle it, you know this, is you divide it into smaller pieces that you can readily solve, and then they become black boxes, and so you reduce the complexity and you add to it.

Steve: Modularity.

Leo: Yeah, modularity builds a complex system. Anyway, I spent some money. And then I thought, well, now that I've spent the money on Claude Code, maybe I should do something with it. So I was trying to think, what do I need? And just to do something simple, I wrote an RSS Reader, a text-based RSS Reader. And this is, by the way, the discussion I had during the show, I said I can't run this. And then it said, oh, that's because of Gatekeeper. So I have removed the quarantine attribute from your RSS Reader binary. Now you can run it without macOS blocking it.

Steve: Wow.

Leo: Thank you, Claude Code. It also, I found some little other issues. For instance, I didn't realize this, but on Linux the configuration file, which is a TOML file, is kept in a different directory on the Mac. So I said, what's going on? So it did - this is its debugging. It went through a whole debugging process. It wrote a debugger. It said, what's the error message you're getting? I pasted it in. It said, oh, duh. I've fixed the config location issue. I've added your API keys to the correct config location. On the Mac it's in an application support folder. I should have known that. Eventually - this is the point. I would have probably figured that out. But I didn't have to.

Steve: And that's exactly my experience, Leo. It is really an accelerant.

Leo: Yes.

Steve: I mean, it allows an expert to just run much more quickly by, you know...

Leo: I don't have to go through manuals. I don't have to...

Steve: Yes. And do Google searching and dig through a bunch of nonsense links of people guessing what the problem is. It's like, no, okay, yeah...

Leo: So this is the GitHub, and it's public on GitHub if you want to look at it. My GitHub handle is leolaporte, and it's the RSS Reader. But the point is, this is not for the general public. I didn't write a general program. I wrote a program that's specifically for what I wanted. It's terminal based, it's very fast, it does AI article summaries, it bookmarks it to Raindrop.io. It does a lot of things. It's just what I wanted. Now, it built it in Rust. It said, you want Python or Rust? I said, oh, well, if you could do it in Rust, go ahead. This is all the Rust code. There's quite a bit of code. It built this in a morning with very little interaction. I interacted with it a little bit, but not a whole lot of interaction. I did some, you know, there was some back and forth.

You know, there are some things I didn't like, that didn't work. So I said, can you do this? As it built it, it used GitHub actions to create binaries that work on Linux and Mac, Intel and Apple Silica. And I didn't even ask it to do that, but it did. It added a Help. It's got a Help feature. It's got a bunch of single keystrokes. It automatically made, I said, hey, is there a way that I could automatically update these RSS feeds every hour? It said sure, let me just set that up for you.

So let me show you the app because it's - first of all, I'll exit out of it. In fact, let me make my - well, I'll do the screen bigger in a second. So this is it. It's RSS Reader. It's loaded in a series of RSS - oops, didn't want to print - RSS things that I had - make it bigger so you can see it, which is not the best UI because, as you can see now, the headlines go off the side of the page. But so I am on - this is stories. If I say no, I never want to have - actually, that one I won't delete. I don't need this in, you know, any of our shows, so I'm going to delete it. Delete this. Going to delete this.

Ah. Governor clears path for robo taxis in New York. So let me hit Enter, and it's going to generate - it goes out to Claude and generates an AI summary. If I want to, I can just hit "O," and it will open it in the browser, so I can read it in the browser. It added that all by itself. I didn't even ask it for that. The navigation is single-key, vim-style navigation. I can refresh the feed. Once I look at a summary - oops, I forgot to hit Enter. Oh, and you see, I said hey, I don't know what's going on when you're generating. Could you put a little - so it said let me put a little hourglass there. Then you'll know. This is the AI generator. Now, this is the thing...

Steve: Oh, my god.

Leo: ...that was specific to me. I save all the articles I want to use to Raindrop. So Capital S saves it to Raindrop. It says what's the tag? I'm going to say that's for TWiT and IM and hit Return. And now it's bookmarked on Raindrop. See, it even put a little raindrop at the bottom. So these are the - so this makes me a very - what I wanted was a very quick way to scan through hundreds of stories. I do this every single day. Look at the headline and then say, yeah, tell me more about that. Yeah, bookmark that. No, delete that. It wrote it. It's done. And it's easy for me to fix it so that, you know, if I want a new feature, I can easily do that. In fact, I'll show you.

We'll go back to Claude Code. And I can say, can you add a key for, I don't know, emailing the story? By the way, it's pretty good on misspellings. And so this is what Claude Code looks like. Let me get rid of the lower third here so you can see it because it's kind of - it uses a lot of fun verbs. It says "fermenting." You can go into - and then it will ask you questions. You can go into plan mode or coding mode. So how would you like to send it? Do you want to open the email app? Default email app? You want to send by SMTP? You want to use an email? I think I'm just going to have it open the app. So I'm going to hit "1," and it will do that. Well, not here.

Okay. And then what content should be included? Oops, I'm sorry. Too many buttons. I need Claude to help me switch the show. Don't worry, Benito, your job is safe. Trust me. What do I want? I want article title, AI summary, full article content. I think I just want - I did it again. Sorry. I just want a "1" here, so we'll just hit "1." Oh, I guess I can check. Oh, it's checkboxes. Oh, that's - yeah, let's do it all. Okay.

Steve: Wow.

Leo: Submit. Thank you. How would you like to send emails with content? Okay, now, submit those answers. So it did a little back-and-forthing.

Steve: And it's crafting right now.

Leo: It's doing it. And it's doing it in Rust, by the way, which I don't know. And I've wanted to write Rust. Now, it could probably do assembly language. It can certainly do Common Lisp. It knows a variety of languages. It's probably best at Python, I would imagine. Python seems to be the native language of a lot of AI. But I thought, well, let's try it with Rust because it'll be memory safe. Type safe. Anyway, we don't have to go on. But you see it's coding right now. It's doing the actual work of implementing email functionality. Which I didn't have built in. So now I will.

Steve: So something turned a corner.

Leo: Yes.

Steve: And the consequence, I mean, this suddenly got - Claude Code got very real. And we have Build With Andrew from, you know, grc.sc/andrew, which is a 30-minute YouTube video, basically from an AI founder who is explaining how to talk to AI, how to explain what you want from an application that you want the AI to write for you. So anyway, it's got interesting gap bridging.

Leo: I would also suggest you can do quite a bit with the free plan. The 20 buck plan will be enough for almost everybody.

Steve: Yes.

Leo: Play with it is the best way because one of the things I've noticed is this stuff is moving so fast that stuff gets out of date right away. I'm sure Andrew's is not out of

date. It's brand new. So, you know, stick with stuff that's brand new. And but I think experimenting is often the best thing. There's something else Darren was telling me about that I was not aware of. Google also has something called Opal, which is designed to use Gemini to do mini AI apps for people who are not technical. It's a no-code version of doing this. And this is free.

So there's other ways to get into this, even if you're not a coder. I think it's probably the case that, even as good as Claude is, it's good if you know a little bit about technology. Oh, by the way, it's done. Email will include the summary if you've generated it, other just the title. Okay. Try it out. So that's how fast it did it. And now if I run RSS Reader, it'll have a new capability. See, at the bottom it says "E" for email.

Steve: Wow.

Leo: And I'll just - let's generate a summary for this. The summary's a little slow. I could probably use a different model that would be a little bit faster. I'm using the most, the heaviest model right now, Opus. But let's email that, and let's see if email works. Should it - yeah. There you go. And I'll just mail this to you, Steve. How about that? How about that? And that's not his address. That's an old address.

Steve: Wow.

Leo: So how about that? Pretty cool, huh?

Steve: Very cool.

Leo: I just added a massive feature that I could never have added in five minutes.

Steve: Yup. And, oh, look at that, I just got your email.

Leo: Isn't that wild?

Steve: Because eM Client is now working.

Leo: Yes. Now, I think there's still going to be lots of room for hand-coded stuff like you do, or even stuff like eM Client. But I think what's changing is a lot of the little stuff, you know, that great Jonathan Coulton song, "Code Monkey," going to write a login. A lot of the stuff that's just kind of rote, you don't need a code monkey for anymore. You just have Claude do it, and then you get the higher level thinking, the overall planning, the architecting. And maybe if you want some fine-tuning or refinement, you do that. So there's still a human in the loop. But I think increasingly - but look, boilerplate code will be written by AI. It's just too easy. And by the way, it writes pretty good code. I mean, everything I've looked at, the code is pretty good, yeah.

Steve: Wow. Very cool.

Leo: Thanks for asking.

Steve: Yes.

Leo: I've been wanting to tell somebody about this.

Steve: Well, and now you've let our listeners know that, well, and I love it because it works perfectly with this Build by Andrew to get started.

Leo: Yeah.

Steve: And then this sense that something, like, is really getting better. It's doing a much better job.

Leo: It's massively better. And that's the other thing is so many people are now into this, there's a lot of resources. There's a wonderful GitHub page called Awesome Claude that has hundreds of resources for using Claude. Claude uses skills. It uses a lot of extra tools. There's a tool called Ralph Wiggum. That's a hysterical - I know, that's a Simpsons character. It's a hysterical tool that you turn on, and you say what the parameter is, like don't come back until there are no more errors. And also if you want optionally can say it, but only try 20 times. You can limit the iterations. But it will keep iterating until it reaches the goal that you set it. So it can, you know, instead of you interacting with it, you just set it off, and it goes. And a lot of people are doing that now. They're running 20 different versions of Claude Code all at the same time. And, ooph. Very interesting.

Steve: Okay. Time for a break.

Leo: Yeah. Sorry, Burke. Burke says, "Boy, Leo, you could tell you've been wanting to talk about this." I apologize. You can edit this out if you want. Let's continue on with Security Now!. Steve?

Steve: So we have listener feedback, although I already shared some little bits that have dribbled in over the last week already. But TJ Asher wrote: "Steve, I am all too familiar with the current state of the move to pure revenue generation by Certificate Authorities around code signing. We were first hit by this with the change to HSM storage" - meaning got to store it in hardware; right? - "two and a half years ago."

He said: "Our corporate policies prohibit allowing multiple users to access a computer under a common ID, which makes sense. Then, because one of our development environments installs some aspects 'per user' under the HKEY_CURRENT_USER tree of the registry, our current licensing of certain add-ons would require an additional license for every possible user on that computer. So we're unable to implement an HSM solution to hold our code signing cert. As a result, we have no option but to store the key in the

Cloud. MS Azure has an option to allow this, but I was informed by our IT group that this costs a minimum of \$30,000 to set up. Yes, that's the number we were told."

Okay. Now, I'll just interrupt TJ's note to mention that Rick Strahl's detailed "How to setup MS Azure code signing" blog posting, which we shared at the top of the show, might be something that TJ would like to show his IT group. They may have some other situation that imposes a \$30,000 cost, but it's difficult to imagine what that might be. Maybe there's a way around that.

In any event, TJ's note continues, writing: "The other option is for our Certificate Authority to host it. But then we need to pay for each and every signature that happens. We have dozens and dozens of files that need to be signed frequently because we release updates every month. This quickly adds up, and you have to pre-pay for the signatures in blocks of 1,000. That fee just went up and is now 26.4 cents per signature, so \$264.00 per 1,000. And NO REFUNDS! Bought too many? Too bad, so sad. We go through enough signatures that we now buy in blocks of 5,000."

Okay. So let me just pause here to remind everyone just how absolutely and utterly insane this has all become. TJ's enterprise that needs to be performing lots of code signing is stuck paying for the privilege of signing its own code on a per-file basis. It should all be a non-issue. They should be able to sign their code just as readily as they compile the code to be signed. But no. By making it increasingly difficult to sign code, for no good reason other than because they can, and by shortening certificate lifetimes, again, because the cabal of certificate authorities vote their own self-interests, the industry's certificate authorities are able to force everyone into a cloud-based service model where our use of our own signing key will be monitored, and we will be charged essentially a fee per signing.

TJ finishes: "The Certificate Authority group has the entire software industry over a barrel, and there is naught we can do about it. Woe help you if you have a problem like we are experiencing now. Need good support? Buy a business account. It's no wonder this change to yearly expiration is happening, because they're going to lose out to free TLS certificates from Let's Encrypt. They need to recoup that revenue somewhere. I can't imagine being a small or solo developer. Regards, TJ Asher."

And then Jay Thompson wrote: "Are you interested in starting a service to issue certs? I put grccerts.com and grccodecerts.com on hold just in case. Signed, Jay."

So first of all, Jay, I very much appreciate your consideration. There are many considerations. But addressing the name of such a service first, if - and it's not going to happen. But if I were to start a certificate authority, I wouldn't tie it to GRC in any way. It would need to have some sort of long-lived neutral name like DigiCert or IdenTrust or VeriSign. Those are good names for a certificate issuing authority.

And I said "first of all" because, you know, there's more. You know the saying "Everyone has their own version of hell." In my case, having anything whatsoever to do with running or in any way managing a certificate authority would definitely be right up there near the top of the most hellacious ways I can imagine me spending the remainder of my life. So thank you, but no thank you. I know that I bitch and moan and carry on about the annoying cabal that has been allowed to form, but at the same time I deeply appreciate that there are people who are able to do what is completely beyond me.

In the early days of GRC I ran a larger organization because I thought I was supposed to. And while my employees may have been happy, I was mostly miserable. Thanks to one rough Monday morning of firings, during which I reduced the company size in half, followed by a great deal of welcome attrition, I wound up with Sue to deal with operational stuff, and Greg to run interference for me with tech support. Which leaves

me mostly completely free to sit in a quiet corner by myself, with elevator music, a PC, mouse, screens, and keyboard. That's my bliss.

But Jay's note brings up the interesting question of the contemporary creation of a new certificate authority. It's not a simple thing, and it would require a concerted effort. But that said, I would imagine that the apparent greediness that is overtaking parts of the certificate authority business might be creating an opening for a well-financed newcomer.

The first problem any newcomer would encounter would be the establishment of their own root certificates into the heart of every single system where their signed certificates would need to be trusted. This might seem like a classic chicken-and-egg problem since you cannot sell any certificate whose signature will not be trusted, not even one, and it'll be difficult to convince the various root store programs to accept any new and unproven and currently unnecessary root certificate without good cause.

Let's Encrypt solved this chicken-and-egg problem by "borrowing" the trust relationship which IdenTrust had already long established. At Let's Encrypt's launch, which is now 11 years ago in 2015, its own root certificate was not present in a single trust store. So, in order to bootstrap trust, Let's Encrypt certificates were cross-signed by IdenTrust's well-trusted root certificate.

The way this worked was slick. Let's Encrypt first created its own new intermediate certificate. This intermediate certificate is what was used to sign all of Let's Encrypt's TLS web certificates which it was issuing. But the signatures made by this intermediate certificate needed to be trusted by all of the world's web clients. To make that happen, Let's Encrypt's new intermediate certificate was co-signed - technically the term is "cross signed" - by Let's Encrypt's newly minted and not yet trusted root certificate, and also by IdenTrust's own already well-trusted root certificate. So two different root certificates trusted, you know, they signed and therefore demonstrated their trust of the intermediate certificate which Let's Encrypt was then able to use to sign the end certificates, the TLS web certificates.

The use of IdenTrust's root certificate to anchor the certificate chain meant that the signatures Let's Encrypt's intermediate certificate was placing on TLS certificates would be trusted from day one by all web clients, since those TLS web certificates "chained up" through Let's Encrypt's intermediate certificate to a root certificate everyone already trusted.

For three years, from 2015 to 2018, Let's Encrypt's certificate trust chain relied solely upon IdenTrust's root cross-signing. And then finally, after three years, in 2018, which I suppose finally after Let's Encrypt had demonstrated the success of their concept, their own operational integrity, and the feasibility of their new ACME automation technology, and I imagine the existing CAs were not happy, but their own root certificate which was named ISRG Root X1 was added to all of the major trust root stores. And then finally, in 2021, three years after Let's Encrypt's root had been added to everyone's root stores, the root certificate that IdenTrust had originally used to cross-sign Let's Encrypt's intermediate certificate itself expired, ending the cross-sign phase and leaving all modern systems trusting Let's Encrypt's own root and the intermediate certificate it had signed.

So a lesson taught by this bit of history is that creating a new trusted certificate-issuing authority is neither quick nor easy, nor should it be. It would require an entity to first demonstrate both their strict commitment to rule-following and their ability to rigorously follow the rules that they set. They need to demonstrate that imbuing their signatures with widespread global trust would not in any way endanger the current status quo. If someone really wished to do so, they could arrange to bootstrap themselves into business the same way Let's Encrypt did. And I doubt that the members of the CA/Browser forum could prevent that from happening, much as they might not wish to

have a powerful new lower-priced, easy-to-use Certificate Authority undercutting their well-established cash-printing business.

And taking the opposite view for a moment, we should all definitely require any upstart newcomer to prove themselves worthy of our trust. There's big money to be made in the certificate issuing business. The bigger the big guys get, the more dead weight overhead they accrue and need to carry. And the costlier their certificates become, the more tantalizing will be the opportunity for newcomers to attempt to get in for a piece of that action. Anyone should have the ability to become a certificate authority in good standing. But as we've often noted, along with the right to print money comes the burden of being very careful whose certificates are signed and thus trusted.

So interesting question, Jay. Being a CA is not for me. I like the way my life is right now. But I really - I can see, you know, Let's Encrypt managed to start, and they were, you know, that was 11 years ago. So I can see an entity deciding that they want in and see an opportunity because I think the flipside of all the grumbling and grouching we're doing is demonstrating that there's some opportunity here for someone who is, you know, serious about it in the long term. But it's not something that you do easily or quickly.

Scott wrote: "Steve, I've listened to you for years for your comments and sage advice about security matters and general comments about IT. Do want to say I appreciate your thoughts on vitamins and, after the latest podcast, have increased my intake of magnesium. Please continue to include the occasional thoughts about how vitamins might prolong our lives. Definitely not a bad diversion from the usual themes. Thanks. Scott."

And I just wanted to mention I put that in here as a placeholder to say, as I said earlier, I received similar sentiments from our listeners. And a couple young listeners, one in particular said, "Hey, supplementary nutrition is not just for older folks." So I appreciate that, and I will share what I find from time to time.

To that end I wanted to update a little bit of the news from last week. Steve Penfold said: "Hi, Steve. Thank you for the info on magnesium last week. I've followed your previous leads on vitamin D3, and now K2, as well, plus the ketogenic way of eating. But I wasn't aware of any of the magnesium stuff." He said: "Your book recommendation caused me to take a look at it on Amazon's site here in the UK. It seems that there have been two updates to 'The Magnesium Miracle' book by Carolyn Dean that you said you read in 2009. A quick summary of these updates.

He said: "First of all, in 2009, the version you read must have been the original 2003 version." I think that's probably likely. He said: "There was an updated version released in 2017 with the same title." And he said: "Additionally, there is a now even newer book, from 2023, billed as 'an up-to-date summary that includes the advances in clinical magnesium research and therapy from the past five years.'" He said: "This is the version that I bought, in Kindle format, for only 3.92," he says, "(equates to just over \$5). Note that the word 'miracle' has been dropped from the title." He remembers me grumbling, you know.

Leo: Good. Good.

Steve: But it's like, it's not a miracle. Stop saying that. Yeah, this book is just titled "Magnesium: The Missing Link to Total Health."

Leo: Well, I'm taking my gram of magnesium now, I've got to say.

Steve: Good.

Leo: It's working. It's working.

Steve: Good. Good, good, good.

Leo: My sleep's been better, too, which is nice.

Steve: Yeah. It does that. And be aware that after some length of time you'll find you no longer...

Leo: No, it's already - that's already happened, yeah.

Steve: Okay.

Leo: I was looking for that, actually, because it's helpful.

Steve: Yes, it is, exactly. It is. Lorrie and I both love it.

Leo: I'll back off a little bit if I need to, but so far, so good.

Steve: Nice. So anyway, that was Steve Penfold, SpinRite owner and Club TWiT member. Joey Albert said: "Thank you, Steve. You started me on 'The Lazarus Project' series yesterday, and it is outstanding." He said: "Rotten Tomatoes meter is 100% fresh! Just bummed it's leaving Netflix this month, the 27th. I have to binge now. Signed Joey."

And Mr. Ron said: "Thanks for the tip about 'The Lazarus Project.' I had never heard of it. I just finished attentively binging it, which really is the only way to follow the plot. It's the most outstanding time travel story I've ever seen."

So I just wanted to mention that Joey's and Mr. Ron's opinion was widely echoed among those who wrote. One listener reminded me of Apple TV's "For All Mankind," saying that he thought it was fabulous. For those who don't know, it's an interesting speculative fiction that extrapolates an alternative history where Russia wins the early stages of the space race...

Leo: Oh, yeah.

Steve: ...by beating the U.S. to the moon. I didn't watch the entire series so I'm unable to offer my own opinion. Lorrie and I watched I think like maybe the first four or five episodes...

Leo: Me, too.

Steve: ...until we had caught up.

Leo: Yeah, I lost interest.

Steve: But it seemed to kind of just be lumbering along and not really much. So I don't really recall it being amazing, but maybe I didn't give it a chance. It does rate an 8.1 on IMDB, which is a good score. But I've also seen, like, you know, anime things rate highly, so it's a matter of who's rating them; right? And that's not me.

So Philip said: "Hi, Steve. Many thanks for the valuable lowdown on reduction in the lifetime of code-signing certs. Does this mean that eventually all software will need to be updated every two years? What does that mean for software for which I've bought a perpetual license, or freeware? What if it's no longer maintained? And I guess that you and I, probably the last two users of PaintShop Pro..."

Leo: No, Paul Thurrott loves it, too.

Steve: Oh, okay, good. He says: "...might have to find something else, at last. Best regards, Philip."

Okay. So Philip's quandary about this was echoed by a number of our listeners, many of whom wrote wondering how shorter code signing certificates would affect the long-term verifiable legitimacy of the code those certificates were used to sign. And right on cue, BleepingComputer posted a story last Wednesday with the headline: "Logitech Options+, G HUB macOS apps break after certificate expires."

Leo: Yeah.

Steve: Whoops! So BleepingComputer...

Leo: They fixed that, by the way.

Steve: Yes. BleepingComputer began their coverage by writing: "Logitech's Options+ and G Hub apps on macOS stopped working after their code-signing certificate expired, leaving users unable to launch them on Apple systems. Options+ is Logitech's input device configuration app, while G HUB is a similar app focused on customizing compatible Logitech G gaming peripherals. Both allow setting app profiles, button remapping, lighting options, scroll wheel, and sensor sensitivity.

"Multiple users reported that Logitech apps on macOS did not load, making custom gestures, mappings, scroll settings unavailable, and forcing them" - oh, the horror - "to use basic input functions. Impacted users expressed their frustration with the sudden loss of productivity-enhancing customizations, while many wasted time re-installing the Logitech apps, trying out Safe Mode, or wiping their configuration files. Eventually, Logitech published a statement on its support portal, admitting that the issue was caused by a certificate that had expired."

Okay. At the same time, I signed GRC's Never10 Windows executable program on Sunday, April 21st, 2019, and the code signing certificate I used to do that then expired on April 4th, 2022. Yet a check on the validity of Never10's code signing certificate today reveals that it remains valid. So what's going on? This brings us to this week's topic, an examination of code signing certificate expiration and the answer to the question "How could Microsoft be issuing three-day code signing certs?"

Leo: I do not know.

Steve: And I'm going to have a cup of coffee.

Leo: I just want to say one thing. I'm very proud. While you were talking, I just submitted two pull requests and did a rebuild on GitHub v0.2.2 of the RSS Reader is now out [indiscernible] email functionality, and I sped up the AI summaries by switching models, plus better error messages. So thank you very much.

Steve: You have had a productive day coding, Leo.

Leo: I feel like a real developer.

Steve: While co-hosting a Security Now! podcast.

Leo: It's pretty amazing. It even uses GitHub actions to build the software to put the binaries up there so people can download it. I think open source software is going to really see a revolution. And what's even more interesting to me is that this means people can write their own custom personal stuff. This was always kind of the goal; right?

Steve: Well, and Apple was - what was the card deck thing that Apple had?

Leo: Apricot. That was the...

Steve: Apricot.

Leo: People have been trying to do this for decades.

Steve: Yes. Yes.

Leo: In fact, I remember John C. Dvorak telling me, what was it, was Morris the guy who - Morrow. George Morrow who did little - remember Morrow computers that were a little bit like the Osbornes? They were suitcase computers?

Steve: Yeah.

Leo: George Morrow told Dvorak, he said, you know, everybody should be writing their own software. Nobody should be using off-the-shelf software. Which was hysterical at the time because, you know, not everybody...

Steve: It was impossible.

Leo: It was impossible. But, I mean, you're not going to write your own word processor video editor. But you might write a lot of little tools, I have been, that make your life better.

Steve: Well, everybody's needs are different. I mean...

Leo: And they could specifically be your needs, yeah.

Steve: And the brilliance of Bricklin's spreadsheet was that it was a programming language.

Leo: VisiCalc, yeah.

Steve: It was, you know, VisiCalc allowed you to put numbers in and do with them what you wanted. And, you know, so it was a type of programming language. And, you know, and there are some databases that have been like that through the years where...

Leo: Sure.

Steve: ...they were really - they really helped you get the job done.

Leo: Yeah. And the other thing that has always been the holy grail is natural language interfaces to computers. You know, and this was, you know, hello, computer. Use the keyboard. Keyboard. How quaint. We've known that this is really the natural way to interact with a computer. Let it do the computer stuff. You do the human stuff. But we haven't had that capability.

Steve: Well, and imagine, Leo, when we can put the AI loose on existing repositories and have it find the bugs. Have it find...

Leo: Yeah. Oh, I think we already - you mentioned this a couple of weeks ago. There's already tools to do that. I think that's going to be a revelation, as well.

Steve: Yeah.

Leo: And as we were saying, I mean, yeah, there are security issues that come up. But you can pretty much be sure that Claude is not going to use strcpy instead of strncpy when it's writing - you're not going to see buffer overflows as much because it's smart. It knows that's a bad idea. Humans forget.

Steve: Yeah. And as I said last week, I even had ChatGPT, when I asked it what was the port number for the MongoDB, it gave it to me, said, and by the way, you should not expose that to the public Internet.

Leo: Isn't that great? I think we're in a brave new world. It's certainly an interesting world. There's no question about that. You're watching Security Now!. This is Steve Gibson. We're so glad you're here. Thanks for watching. And let's go on with the show. Steve?

Steve: Okay. So the title for today's podcast was inspired by the sentence that Rick Strahl casually dropped into his blog posting in passing. He noted: "The certificates issued by Microsoft are very short lived, with expirations that last only three days, to aggressively thwart invalid signing attacks in case a certificate is compromised." Okay. Now, this raises the obvious question: How can it possibly be that Microsoft would be using code signing certificates that only last for three days before they expire? The answer to that question brings us to a fundamental difference between the traditional web server authentication TLS certificates, which we're all by now intimately familiar with, and code signing certificates which we've spent considerably less time exploring in the past. So exactly what are the differences between these two?

In the case of a web server's TLS certificate, our goal, the purpose is to validate and authenticate the identity of a remote web server during a real-time transaction right now. We need to be assured that the remote server we have just this moment connected to using its DNS-provided IP address is, in fact, the server we expect. DNS could have been compromised to lead us astray, or our Internet packet traffic could have been intercepted and diverted to a malicious web server.

So to do this, we need to verify that the certificate we've just received over the connection we've just established matches the domain we intend to connect to and that the certificate is valid, not expired, in good standing, not revoked, and was signed by a certificate authority whose signatures we trust. If all of those things are true, we would have every reason to believe that we're connecting to a web server serving the domain we intend.

So now look at code signing. What about code signing? The assurances we seek from signed code are obviously very different from the application of TLS web certificates. We want to ascertain two things from the signature of any signed code: We want to determine the verifiable identity of the entity that signed the code, and we want to verify that not a single bit of the code that was signed has been altered since its signing. And that's it. That's the entire purpose of signed code. Who signed it, and nothing has changed since.

We understand the general reason why certificates have expiration dates. While I complain a lot about certificate lifetimes being so short that their renewal becomes burdensome, at the same time it would be somewhat unnerving to be issued a trusted certificate that never expired. Yikes! If that certificate were to ever get loose at any time ever, bad guys could abuse its trust, potentially forever. GRC has a code signing certificate stored, as they must all be now, in a SafeNet 5110 USB eToken. And it's actually sort of comforting to know that it comes with a drop-dead date after which it will

become useless to anyone. If it didn't have that, I would need to wipe its contents and then probably still smash it into tiny bits to make absolutely sure that it could never be reused once I was finally finished with ever needing it again. You know, I mean, it has to be completely destroyed.

But what about the code that it was used to sign? Let's take the Never10 for Windows executable I mentioned before. If you're curious, you can go to GRC and download that executable right now, Never10.exe, to see for yourself. I signed that executable on Sunday, April 21st, 2019 using a code signing certificate that still had very nearly three years of life left on it, since it would expire on April 4th, 2022. Signed on April 21st, 2019, with a certificate that's expiring on April 4th, 2022. That did expire, had to, on April 4th, 2022. At the time I signed that code, the certificate was in good standing. It was issued to my company, Gibson Research Corporation, by DigiCert. The signing process meant that an unspoofable cryptographic hash was taken of the code, the Never10 code; whereupon the private key I was in possession of, because at this time and still today I own my own code signing private key, it would be used to sign the hash. And GRC's certificate that was issued by DigiCert containing the matching public key was affixed to the end of the code.

From that moment on, anyone who obtained that Never10 code could check its certificate to see that the certificate was validly issued by DigiCert, a certificate authority that has carefully earned everyone's trust. The signature of the code's original hash could be verified using the public key contained in GRC's certificate, and that validly signed hash could be compared with a fresh hash of the code taken right then to verify that not a single bit of the original code had been changed after it was signed. Remember the two assertions that are made through code signing: The identity of the certificate that performed the signing, in this case Gibson Research Corporation; and that since the time of the signing, not a single bit has changed.

Okay. Now jump forward to 2026. There's still a Never10 executable program that can be downloaded from GRC. And not a single bit of that code has been changed since the day it was signed in April of 2019. Yes, the certificate that was used to perform the signing expired three years after the signing, which is almost four years ago, in April of 2022. But do we care? The signature accompanying the code remains valid. The certificate that's attached still contains a public key that can be used to verify that not a single bit has changed since the moment it was originally signed, and Gibson Research Corporation's name is carried in the attached certificate, all of which was signed by DigiCert.

Here's what's common between TLS and code signing certificates: In both cases, the only requirement is that the certificate is valid at the time of its use. So in the case of TLS, that means it must be valid and remain valid every time a web browser initiates a new connection, and that certificate is offered up as proof of the remote server's identity. Connecting to the server is the time of the certificate's use.

But in the case of code signing, the only requirement is that the certificate used to sign the code be valid at the time the code is signed. Since the only thing code signing is asserting is the identity of the signer and that nothing has changed since, requiring that the certificate be valid at the instant of the signing is sufficient. And now we can see why and how Microsoft's Azure code signing uses certificates having a very short life of three days. Technically, it could be as short as an hour. But creating certificates is not without overhead. So I imagine they probably cache any certificates they've created for a couple of days in case the same signer returns with more signatures that they need signed, or more code they need signatures signed for.

But there's an exploit we haven't addressed. What's to keep a bad guy who manages to get their mitts on someone else's expired code signing certificate from using that

certificate to sign their malicious code? The signing certificate may have expired, but what's the enforcement mechanism for its expiration? We might suggest that the PC used to perform the signing would examine the certificate and see that it had expired. Okay. The bad guys know that their stolen certificate has expired. So they simply turn back the clock on the signing PC that they're using to a point where their certificate is valid. Now the PC believes that the certificate is valid and in good standing. It has no way of knowing what day it is.

The obvious answer to this dilemma is for anyone who might be relying upon that certificate to examine for themselves the signing certificate's expiration date and time, just as they would for a "real-time" TLS certificate, and refuse to trust anything signed by any certificate that has expired. Okay, but then we have a new problem. As we've seen, what we really intend for code signing is for any code that's signed by a certificate that is valid at the time of the signing to forever hence be judged as validly signed. So how do we accomplish that?

Introducing the TSA, a different kind of TSA. This is the Timestamp Authority. A Timestamp Authority is a trusted third party. It's typically a certificate authority and is often, but not necessarily, the same CA who provided the signing certificate in the first place. It is a service that CAs offer. During the code signing process, once the code has been signed, the signature, after it's been signed, that signed signature is itself hashed and forwarded to a timestamp authority, the hash is forwarded, and it's bundled with a UTC-format timestamp, and that package is signed with the timestamp authority's private key. They then return this signature along with their own TSA certificate containing their public key. The result is a countersignature containing a verifiable timestamp.

The result of all these machinations is that the final signed code actually contains two certificates: the code-signer's own certificate indicating their identity and the validity time window of their certificate; and a signing timestamp that can be verified using the timestamp authority's certificate, which is also attached. So now we have exactly what we want. The signing certificate's validity window, from the not-valid-before to the not-valid-after times, is enforced by an unspoofable timestamp provided in real time on the fly at the moment of signing by a third-party timestamping service whose own certificate, their public certificate, is also attached to allow their timestamp to be verified. It's because GRC has always signed its code with the aid of a timestamping service that the validity of our apps never expires, even long after the certificate that was used to sign them is long gone.

So what happened with Logitech? The truth is, we don't know because we can't tell from what they've said. They said that a certificate expired. But we don't definitely know what certificate expired. Adding a timestamp to executable code, and to libraries and whatever you need code signed, is now so routine that I'm a little skeptical that they could have actually somehow failed to do that. I mean, it's built in. Timestamping everything ought to be just, I mean, like completely in the core of whatever signed their executable code.

I suspect it's more likely that they have some sort, Logitech being who they are, some sort of their own installer or patcher or updater or who knows what, where they were using their own certificates internally in some fancy system of their own design, and they tripped over their own tail. To me that seems more likely.

It's important to appreciate that it's only commercial certificate authorities who arbitrarily enforce short expiration policies. When you're creating your own certificates for your own internal purposes, you can set whatever expiration date you like. So someone at Logitech may have created a 25-year certificate back in 2001, figuring that the system they're using it for would be replaced long before that certificate could expire. But we all know how that goes; right? So after a few years, everyone completely forgot about it and

never thought about it again until - whoopsie!! - 25 years had flown past, and that long-lived certificate surprised everyone by reaching its end-of-life date and expiring. To me that seems the most plausible explanation. But again, until more is known from Logitech, there's no way to tell.

In any event, now everyone knows exactly what goes on with code signing certificates, and how the static assertions they're designed to make differ from the real-time assertions made by TLS web certificates. And it should be clear how Microsoft's Azure cloud code signing service is able to sign with three-day lifetime certificates. Those signatures are immediately timestamped while that short-lived code signing certificate is valid. After that, the certificate's expiration doesn't matter. It can expire, and no one cares.

Copyright (c) 2014 by Steve Gibson and Leo Laporte. SOME RIGHTS RESERVED

This work is licensed for the good of the Internet Community under the Creative Commons License v2.5. See the following Web page for details:
<http://creativecommons.org/licenses/by-nc-sa/2.5/>